

Course code : **CSE3009**
Course title : **No SQL Data Bases**
Module : **1**
Topic : **4**

Achieving horizontal scalability with database sharding

Objectives

This session will give the knowledge about

- Achieving horizontal scalability with database sharding

Data Storage Limitations

The amount of data needed to run the business exceeds the current environment and some mechanism for breaking the information into reasonable chunks is required.

Organizations and systems that reach this capacity can use automatic database **sharding** as a means to continuing to store data while minimizing system downtime.

Sharding: breaking a database into chunks called shards and spreading the chunks across a number of distributed servers

Data Storage Limitations

Sharding has become a highly automated process in both big data and fault-tolerant systems.

Consider, you've created a website that allows users to log in and create their own personal space to share with friends. They have profiles, text, and product information on things they like (or don't like). You set up your website, store the information in a MySQL database, and you run it on a single CPU. People love it, they log in, create pages, invite their friends, and before you realize it your disk space is 95% full.

What do you do?

Data Storage Limitations

The process of moving from a single to multiple databases can be done in a number of ways; for example:

1. You can keep the users with account names that start with the letters A-N on the first drive and put users from O-Z on the new system.
2. You can keep the people in the United States on the original system and put the people who live in Europe on the new system.
3. You can randomly move half the users to the new system.

Data Storage Limitations

In option 1, if a user changes their name, should they be automatically moved to the new drive?

In option 2, if a user moves to a new country, should all their data be moved?

If people tend to share links with people near them, would there be performance advantages to keeping these users together?

What if people in the United States tend to be active at the same time in the evening?

Would one database get overwhelmed and the other be idle?

What happens if your site doubles in size again?

Do you have to continue to rewrite your code each time this happens?

Do you have to shut the system down for a weekend while you upgrade your software?

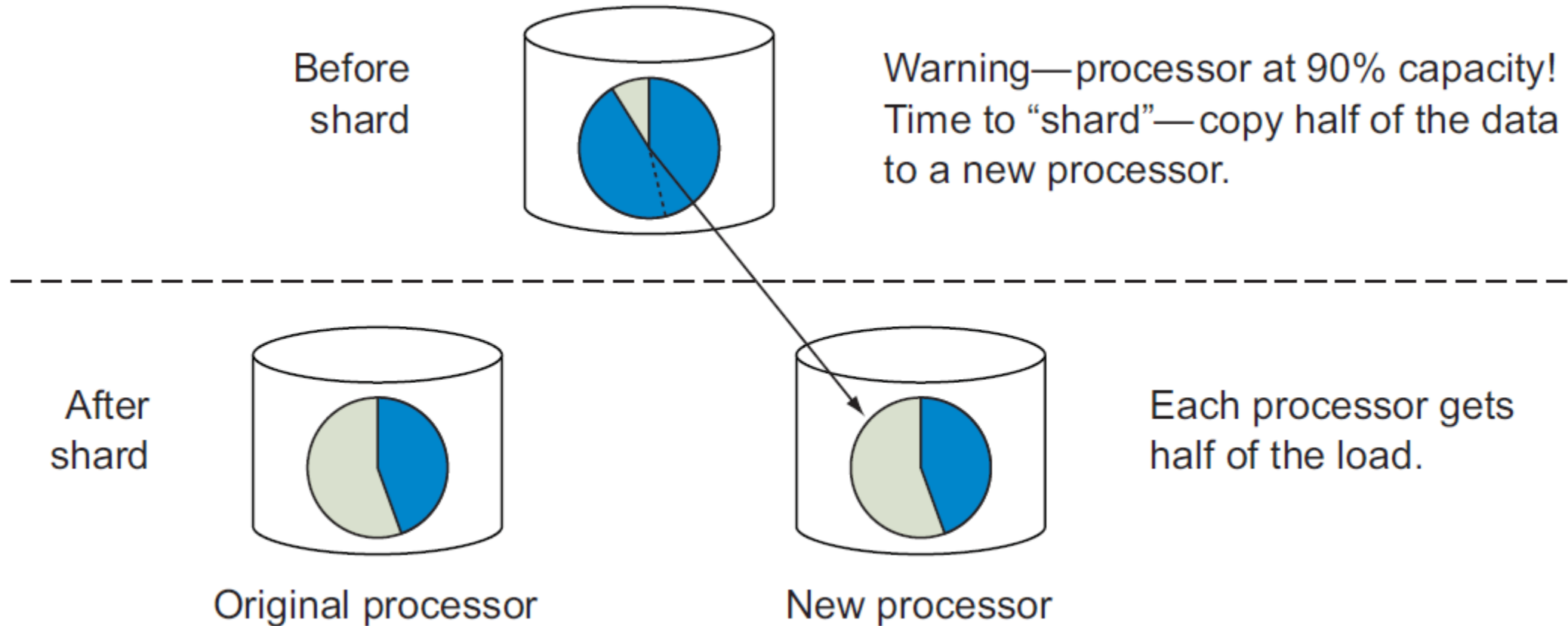
NoSQL Sharding

You can see that there are many questions and trade-offs to consider as your database grows.

SQL systems have been noted for having many ways to allow you to grow your database without ever having to shut down your servers.

Keeping your database running when there are node or network failures is called **partition tolerance** - a new concept in the NoSQL community

NoSQL Sharding



NoSQL Sharding

Original Table

CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin
2	Bob	Best	Boston
3	Carrie	Conway	Chicago
4	David	Doe	Denver

Vertical Shards

VS1

CUSTOMER ID	FIRST NAME	LAST NAME
1	Alice	Anderson
2	Bob	Best
3	Carrie	Conway
4	David	Doe

VS2

CUSTOMER ID	CITY
1	Austin
2	Boston
3	Chicago
4	Denver

Horizontal Shards

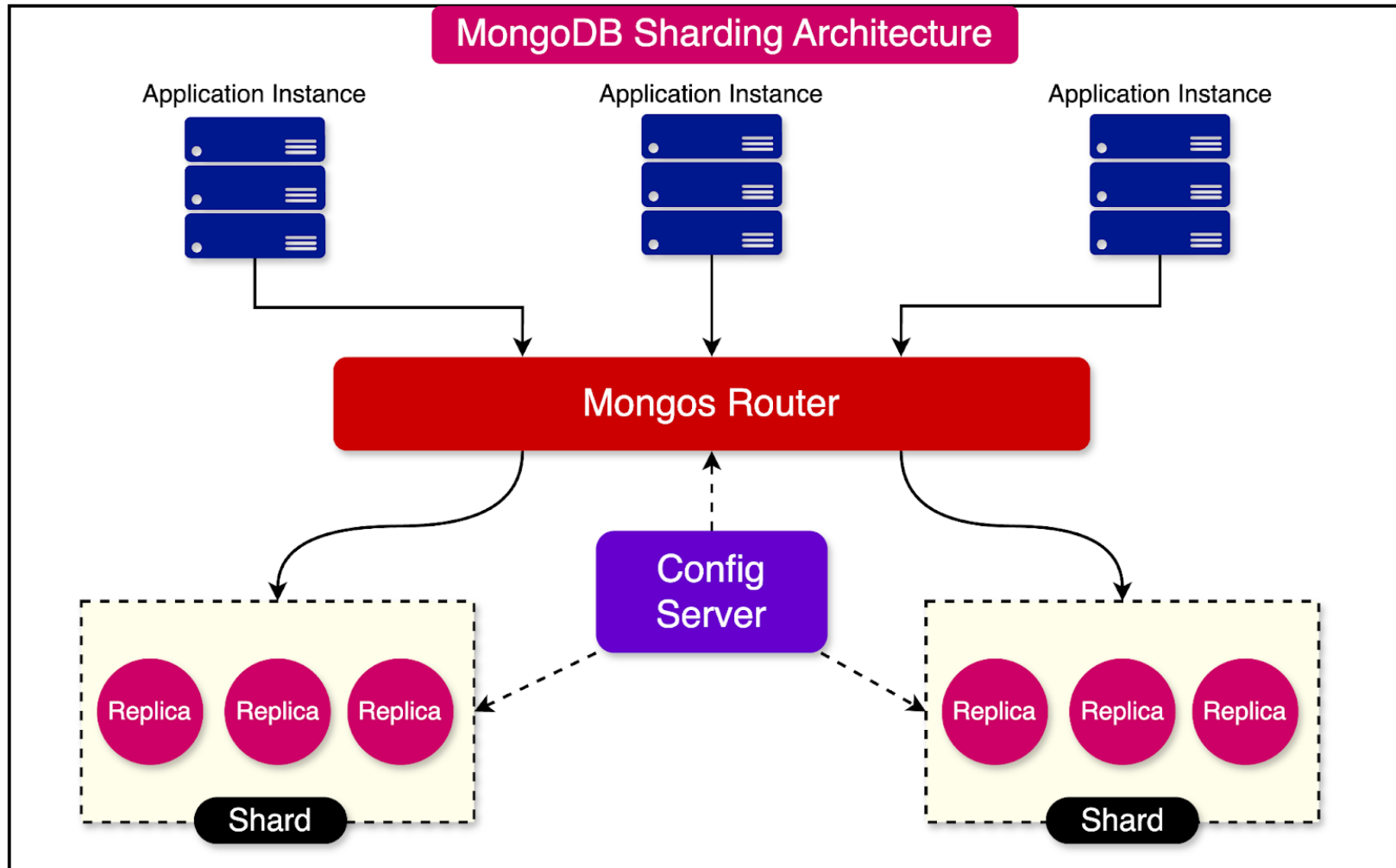
HS1

CUSTOMER ID	FIRST NAME	LAST NAME	CITY
1	Alice	Anderson	Austin
2	Bob	Best	Boston

HS2

CUSTOMER ID	FIRST NAME	LAST NAME	CITY
3	Carrie	Conway	Chicago
4	David	Doe	Denver

NoSQL Sharding



NoSQL Sharding

Sharding is performed when a single processor can't handle the throughput requirements of a system.

When this happens you will want to move the data onto two systems that each take half the work.

Many NoSQL systems have automatic sharding built in so that you only need to add a new server to a pool of working nodes and the database management system automatically moves data to the new node.

Most RDBMSs don't support automatic sharding.

Steps for Database Sharding

Step-1: Determine Sharding Strategy:

- **Range Sharding:** Data is divided based on ranges of a specific key. For example, customer IDs 1-1000 go to Shard 1, 1001-2000 to Shard 2, etc.
- **Hash Sharding:** A hash function is applied to a key to determine the shard. This ensures an even distribution of data across shards.
- **Directory-Based Sharding:** A lookup table (directory) maps each key to a shard. This provides flexibility but adds complexity.

Steps for Database Sharding

Step-2: Design the Shard Key:

- Choose a key that evenly distributes data across shards to avoid hotspots.
- The key should be part of most queries to direct them to the appropriate shard quickly.

Steps for Database Sharding

Step-3: Setup Infrastructure:

- Deploy multiple database servers to host the shards.
- Ensure that servers are configured for high availability and redundancy.

Steps for Database Sharding

Step-4: Data Partitioning:

- Implement the chosen sharding strategy to partition the existing data.
- Migrate data to the appropriate shards based on the shard key.

Steps for Database Sharding

Step-5: Modify Application Logic:

- Update the application to route queries to the correct shard based on the shard key.
- Ensure that write operations are directed to the shard holding the relevant data subset.

Steps for Database Sharding

Step-6: Implement Shard Management:

- Monitor and manage the shards for load balancing and resource optimization.
- Add new shards dynamically as the data grows to maintain performance and scalability.

Steps for Database Sharding

Step-7: Handle Cross-Shard Queries:

- Implement mechanisms to handle queries that span multiple shards.
- Use a middleware or query routing layer to combine results from different shards if necessary.

Steps for Database Sharding

Step-8: Ensure Consistency and Reliability:

- Implement mechanisms for data replication and backup to ensure data consistency and reliability across shards.
- Use distributed transactions carefully to maintain ACID properties where necessary.

Sharding in Real-World Applications

- **Social Media Platforms:** Social media companies use Sharding to manage massive amounts of user-generated content, such as posts, photos, and videos. Sharding ensures fast access to user data and high availability.
- **E-commerce:** Online retailers employ sharding to handle large catalogs of products and accommodate high website traffic. Sharding is essential for managing order data and inventory.
- **Gaming:** Online gaming platforms use Sharding to distribute game state data and player profiles. Sharding ensures low-latency gaming experiences, even in globally distributed multiplayer games.

Sharding in Real-World Applications

- **Financial Services:** Financial institutions rely on Sharding to manage vast amounts of transaction data, customer records, and financial histories. Sharding enhances performance and data security.
- **Content Delivery Networks (CDNs):** CDNs utilize geo-based sharding to cache and deliver web content efficiently to users worldwide. Data is distributed to edge servers close to end users to reduce latency.
- **IoT and Telemetry:** Internet of Things (IoT) platforms leverage sharding to manage the massive influx of data generated by sensors and devices. Sharding helps process and analyze telemetry data in real-time.

Factors to Consider Before Sharding

- **Data Distribution:** Ensure that the chosen shard key distributes data evenly to avoid hotspots or imbalances in shard sizes.
- **Application Complexity:** Sharding increases the complexity of the application logic. Ensure your team is equipped to handle the added complexity in terms of development, deployment, and maintenance.
- **Cross-Shard Operations:** Be aware of the challenges in handling transactions and queries that span multiple shards. Implement strategies for managing these operations efficiently.

Factors to Consider Before Sharding

- **Rebalancing Data:** Plan for data rebalancing as your dataset grows or your access patterns change. Rebalancing can be resource-intensive and should be managed carefully.
- **Consistency and Reliability:** Consider how sharding affects data consistency and reliability. Ensure you have mechanisms in place to handle replication, backup, and recovery across shards.

Summary

This session will give the knowledge about

- Achieving horizontal scalability with database sharding